CHAPTER 10

Vertical vs Horizontal Scaling

Part II · Building Blocks — the two ways to grow, and the ceiling that forces the hard one.

LEARNING OBJECTIVES

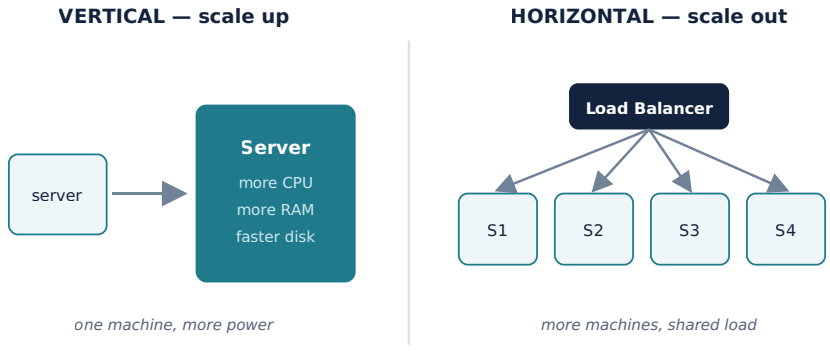
- Define vertical scaling (scale up) and horizontal scaling (scale out) and their trade-offs.
- Explain why every large system eventually scales out, and why statelessness is what makes it possible.
- Explain autoscaling and elasticity, and what they require.
- Identify why the data tier is the hard tier to scale — the bridge to Part III.
- Follow a pragmatic, step-by-step scaling path instead of over-building on day one.

REAL-WORLD ANALOGY — A BIGGER TRUCK VS MORE TRUCKS

A delivery company with too many parcels has two choices. It can buy one **bigger truck** that carries more (that's **vertical** scaling — a more powerful single machine). Or it can buy **more trucks** and split the deliveries among them (that's **horizontal** scaling — more machines sharing the load). The bigger truck is simple but there's a limit to how big a truck you can buy — and if it breaks down, everything stops. More trucks can grow almost without limit and survive one breaking down, but now you need a dispatcher to coordinate them. That dispatcher is your load balancer.

10.1 Two ways to grow

We've been building toward this since Chapter 1. When one machine can no longer keep up — too many requests, too much data — you scale. There are exactly two directions, and knowing when to use each is a core design skill.



10.2 Vertical scaling (scale up)

Give the single machine more power: more CPU cores, more RAM, a faster disk, more network bandwidth.

Advantages	Disadvantages

Simple — no code or architecture changes. No distributed-systems complexity. Strong consistency stays easy (one machine, one copy of data).	A hard **ceiling** — you can only buy so big. Cost grows **non-linearly** (top-end hardware is disproportionately expensive). Still a **single point of failure**. Often needs downtime to upgrade.

Vertical scaling is the right *first* move: it's cheap in engineering effort and buys real time. It's also often the pragmatic choice for databases, which are genuinely hard to distribute (Part III).

10.3 Horizontal scaling (scale out)

Add more machines and share the work across them, coordinated by a load balancer (Chapter 7).

Advantages	Disadvantages
Near-**unlimited** growth — just add machines. **No single point of failure** with several nodes. Uses cheap commodity hardware. Scales incrementally and **elastically** with demand.	Real **complexity**: needs load balancing, **stateless** servers (Chapter 6), and — the hard one — distributing data while keeping it consistent (Part III). Harder to operate and debug.

KEY INSIGHT — STATELESSNESS IS WHAT MAKES SCALE-OUT POSSIBLE

You can only add identical app servers freely because they're **stateless** (Chapter 6): any server can handle any request, so the load balancer can spread traffic and replace dead nodes at will. The moment a server holds per-user state in memory, scale-out breaks (sticky sessions, lost state). This is why "keep the app tier stateless, push state to a shared store" is the rule that unlocks horizontal scaling.

10.4 Autoscaling and elasticity

In the cloud, horizontal scaling becomes **elastic**: the system automatically adds instances when load rises and removes them when it falls. **Autoscaling** can be *reactive* (add servers when CPU or QPS crosses a threshold), *scheduled* (scale up before a known busy period), or *predictive* (forecast demand). This is how spiky traffic (Chapter 5) is served without paying for peak capacity around the clock.

WHAT AUTOSCALING REQUIRES

Elasticity only works if the pieces from earlier chapters are in place: **stateless** servers (so new ones can join instantly), **fast startup**, **health checks** and a **load balancer** (so new instances receive traffic and dead ones are dropped). Autoscaling is the payoff of doing the earlier building blocks correctly.

10.5 The hard tier: scaling data

KEY INSIGHT — APPS SCALE OUT EASILY; DATA DOES NOT

The stateless app tier scales out almost for free — add servers behind the load balancer. But you **cannot simply clone a database**, because every copy must agree on the data. Distributing data across machines while keeping it correct is *the* hard problem of large

systems, and it's exactly what **Part III** is devoted to: **replication** (copies for reads and safety), **partitioning / sharding** (splitting data across machines for writes and size), and the consistency trade-offs (**CAP**) that come with them.

10.6 A pragmatic scaling path

Real systems don't leap to a thousand shards on day one. They grow in steps, adding complexity only when the numbers demand it (Chapter 1's warning against premature optimization):

```
1. One server                (simple; fine to start)
2. Scale UP  (bigger box)     -> cheap, buys time
3. Separate tiers            -> app / database / cache split (Ch 6)
4. Scale the app tier OUT    -> many app servers behind an LB (Ch 7)
                                + cache (Ch 8) + CDN
5. Scale reads               -> database read replicas (Part III)
6. Scale writes & data size  -> sharding / partitioning (Part III)

Move to each step only when the previous one is no longer enough.
```

THE JUDGMENT

Each step down this path adds capability *and* complexity. Jumping straight to step 6 for an app with a thousand users is over-engineering — pure cost, no benefit. The skill is reading your Chapter 5 numbers and moving exactly as far down the path as the load requires, and no further.

Common mistakes

WATCH OUT

- Reaching for horizontal scaling and sharding when a bigger machine would have done the job for years.
- Trying to scale out an app tier that still holds **state** in memory — it won't work cleanly.
- Assuming the database scales like the app tier — ignoring that copies must stay consistent.
- Building autoscaling without stateless servers, health checks, or fast startup.
- Treating vertical scaling as "wrong" — it's often the correct, cheapest first step.
- Forgetting that top-end vertical hardware costs rise non-linearly.

Interview questions

1. Define vertical and horizontal scaling. Give the main pros and cons of each.
2. Why do large systems eventually scale out rather than up?
3. Why is statelessness a prerequisite for horizontal scaling?
4. What is autoscaling, and what does it require to work?
5. Which tier is hardest to scale, and why?
6. Walk through a pragmatic scaling path from one server to a sharded system.

7. When is vertical scaling the *right* choice?

Hands-on exercises

1. For a service hitting its single server's CPU limit, list one vertical and one horizontal option and a trade-off of each.
2. Identify what must be true about your app servers before you can autoscale them.
3. Given traffic that is 10× higher for two hours each evening, describe an autoscaling policy.
4. Explain why you can add app servers freely but not database servers.
5. Place your capstone system on the six-step scaling path — which step does your Chapter 5 scale require?

Mini project

PROVE SCALE-OUT WITH A LOAD TEST

Run one instance of a small app behind a load balancer and use a load-testing tool to push requests until latency degrades (recall the utilization curve, Chapter 5). Record the throughput ceiling. Now add a second and third identical instance behind the same load balancer and repeat. Watch the throughput rise roughly with the number of instances — horizontal scaling, measured with your own hands. Note what would break if the app kept session state in memory.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE SCALING PLAN

Write your platform's **scaling plan** along the six-step path. Where do you scale up first? When do you scale the app tier out (and confirm it's stateless)? Define an **autoscaling** policy for your evening peak from the Chapter 5 estimates. Then mark the point where the plan hits the wall — the **data tier** — and note that replication and sharding (Part III) are the next tools you'll need. This plan is the natural handoff from Part II into Part III.

Chapter summary

- **Vertical scaling** (bigger machine) is simple and a good first step, but has a hard ceiling, rising costs, and remains a single point of failure.
- **Horizontal scaling** (more machines) grows almost without limit and removes single points of failure, at the cost of real distributed-systems complexity.
- Scale-out of the app tier works only because it's **stateless**; externalize state to a shared store.
- **Autoscaling** makes horizontal scaling elastic, but requires stateless servers, fast startup, health checks, and a load balancer.
- The app tier scales out easily; the **data tier is the hard problem** because copies must stay consistent — the subject of Part III.

- Follow a **pragmatic path**: up, then separate tiers, then app-tier out, then read replicas, then sharding — each step only when needed.

COMING NEXT — PART III BEGINS

Chapter 11 — Relational Databases & SQL. Part II gave you the building blocks that scale easily. Part III confronts the hard tier — data. We start at the foundation of storage: the relational model, SQL, and the guarantees that make databases trustworthy, before we learn to replicate and shard them.